

Hi,

In this document you'll find some tips on how to implement various parts announced in the homework.

Part 1 - data cleansing job

Exercise 1 - data classification

Scala/Java

1. First, create a new Kafka input DataFrame:

```
val dataFrame = sparkSession.readStream
  .format("kafka")
  .options(KafkaInputConnectionOptions)
  .load()
```

2. Read the data and convert the input string into a structure

```
val schemaForDataValidation = StructType(Array(
  StructField("user_id", LongType), StructField("visit_id", StringType)
))

val query = dataFrame
  .select(
    $"key".cast("string"),
    functions.from_json(
      $"value".cast("string"), schemaForDataValidation
    ).as("validation_data"),
    $"value".cast("string")
  )
```

At this point you can access the value with `validation_data.user_id` or `validation_data.visit_id` expressions.

3. Use an if-else statement to create a new column called *topic*:

```
.withColumn("topic", functions.when(
  ($"validation_data.user_id" === 0 ||
   $"validation_data.visit_id".isNull),
  "invalid_data").otherwise("valid_data")
)
```

Apache Spark Kafka sink will use the value from the topic column to identify the target topic for the given row.

4. Select the key, value and topic columns:

```
.select("key", "value", "topic")
```

Those are the columns expected by the Kafka writer.

5. Create a Kafka writer and start the query:

```
val writeQuery = query.writeStream
  .option("checkpointLocation", "/tmp/checkpoint/job1")
  .format("kafka")
  .option("kafka.bootstrap.servers", "localhost:29092")

writeQuery.start().awaitTermination()
```

Python

1. First, create a new Kafka input DataFrame:

```
input_data_stream = spark.readStream \
  .option("kafka.bootstrap.servers", "localhost:29092") \
  .option("subscribe", kafka_input_topic()) \
  .option("startingOffsets", "EARLIEST") \
  .format("kafka") \
  .load()
```

2. Read the data and convert the input string into a structure

```
schema_data_validation = StructType([
  StructField("user_id", LongType()), StructField("visit_id",
StringType())
])

query = input_data_stream.select(
  functions.col("key").cast("string"),
  functions.from_json(
    functions.col("value").cast("string"),
    schema_data_validation
  ).alias("validation_data"),
  functions.col("value").cast("string")
)
```

At this point you can access the value with `validation_data.user_id` or `validation_data.visit_id` expressions.

3. Use an if-else statement to create a new column called *topic*:

```
.withColumn("topic", functions.when(
  ((functions.col("value.user_id") == 0) |
   functions.col("value.visit_id").isNull())
, "invalid_data").otherwise("valid_data"))\
```

Apache Spark Kafka sink will use the value from the topic column to identify the target topic for the given row.

4. Select the key, value and topic columns:

```
.select("key", "value", "topic")
```

Those are the columns expected by the Kafka writer.

5. Create a Kafka writer and start the query:

```
write_query = query.writeStream\
  .option("checkpointLocation", checkpoint_location_for_job('job1'))\
  .format('kafka')\
  .option("kafka.bootstrap.servers", "localhost:29092")

write_query.start().awaitTermination()
```

Exercise 2 - data deduplication

Scala/Java

1. First, create a new Kafka input DataFrame. You can use the snippet from the previous exercise.

2. Change the schema used for data validation. Add an `event_time` column to it.

```
val schemaForDataValidation = StructType(Array(
  StructField("user_id", LongType), StructField("visit_id", StringType),
  StructField("event_time", TimestampType)
))
```

3. After the `withColumn("topic")` but before the `select("key", ...)` expressions, add a new one that will drop the duplicates:

```
.withColumn("event_time", $"validation_data.event_time")
.withColumn("user_id", $"validation_data.user_id")
.withWatermark("event_time", "10 minutes")
.dropDuplicates("user_id", "event_time")
```

At this moment Apache Spark uses the `user_id` and `event_time` to detect duplicates, and `event_time` to handle the deduplication buffer.

4. Select the key, value and topic columns:

```
.select("key", "value", "topic")
```

5. Use the same Kafka writer as previously.

Python

1. First, create a new Kafka input DataFrame. You can use the snippet from the previous exercise.

2. Change the schema used for data validation. Add an `event_time` column to it.

```
schema_data_validation = StructType([
    StructField("user_id", LongType()),
    StructField("visit_id", StringType()),
    StructField("event_time", TimestampType())
])
```

3. After the `withColumn("topic")` but before the `select("key", ...)` expressions, add a new one that will drop the duplicates:

```
.withColumn("event_time", functions.col("validation_data.event_time"))\
.withColumn("user_id", functions.col("validation_data.user_id"))\
.withWatermark("event_time", "10 minutes")\
.dropDuplicates(["user_id", "event_time"])\
```

At this moment Apache Spark uses the `user_id` and `event_time` to detect duplicates, and `event_time` to handle the deduplication buffer.

4. Select the key, value and topic columns:

```
.select("key", "value", "topic")
```

5. Use the same Kafka writer as previously.

Exercise 3 - data enrichment

Scala/Java

1. Before creating the Kafka input DataFrame, create a JDBC one that references the enrichment dataset:

```
val deviceReferenceDataset = sparkSession.read.format("jdbc")
    .options(JdbcReferenceDatasetConnectionOptions).load()
```

2. Adapt the data schema:

```
val schemaForDataValidation = StructType(Array(
  StructField("user_id", LongType),
  StructField("visit_id", StringType),
  StructField("event_time", TimestampType),
  StructField("technical", StructType(Array(
    StructField("device", StructType(Array(
      StructField("type", StringType)
    )))
  )))
))
```

The device column will be used in the join. So far we don't care about the potential data quality issue that might invalid the join for some cases.

3. Keep the reading and deduplication part as in the previous exercise. Modify the code only before writing the data.

4. Join the streaming DataFrame with the enrichment dataset by simply invoking the join(...) method:

```
val enrichedRecordsToWrite = recordsToWrite
    .join(deviceReferenceDataset,
        deviceReferenceDataset("device_type") ===
        recordsToWrite("validation_data.technical.device.type"),
        "leftOuter"
    )
    .select(
        $"key",
        $"topic",
        functions.to_json(functions.struct($"value", $"full_name")).as("value")
    )
```

At this point it's not important to build the final output structure. That's why I'm simply combining the raw data with the enrichment information.

5. Use the same Kafka writer as previously.

Python

1. Before creating the Kafka input DataFrame, create a JDBC one that references the enrichment dataset:

```
device_reference_dataset = spark.read.jdbc(  
    url="jdbc:postgresql:bde",  
    table="bde_schema.devices",  
    properties={'user': 'bde_user', 'password': 'bde_password'}  
)
```

2. Adapt the data schema:

```
schema_data_validation = StructType([  
    StructField("user_id", LongType()),  
    StructField("visit_id", StringType()),  
    StructField("event_time", TimestampType()),  
    StructField("technical", StructType([  
        StructField("device", StructType([  
            StructField("type", StringType())  
        ]))  
    ]))  
)
```

The device column will be used in the join. So far we don't care about the potential data quality issue that might invalid the join for some cases.

3. Keep the reading and deduplication part as in the previous exercise. Modify the code only before writing the data.

4. Join the streaming DataFrame with the enrichment dataset by simply invoking the join(...) method:

```
enriched_dataset_to_write = query.join(  
    device_reference_dataset,  
    device_reference_dataset["device_type"] ==  
    query["validation_data.technical.device.type"],  
    "left_outer")\  
.select(  
    functions.col("key"), functions.col("topic"),  
    functions.to_json(functions.struct("value", "full_name"))  
    .alias("value")  
)
```

At this point it's not important to build the final output structure. That's why I'm simply combining the raw data with the enrichment information.

4. Use the same Kafka writer as previously.

Exercise 4 - data preparation

Scala/Java

1. Read the Kafka and enrichment dataset as before:

```
val deviceReferenceDataset = sparkSession.read.format("jdbc")
    .options(JdbcReferenceDatasetConnectionOptions).load()

val dataframe = sparkSession.readStream
    .format("kafka")
    .options(KafkaInputConnectionOptions)
    .load()
```

2. Define the case classes for the input data and the schema with the help of SchemaReflection

```
object EventLog {
  val Schema = SchemaReflection
    .schemaFor[EventLog].dataType.asInstanceOf[StructType]
}
```

3. Perform the data cleansing by converting the JSON string into a struct and casting it into the case class you defined previously. Call the data cleansing function on top of it with the map transformation:

```
val recordsToWrite = dataframe
    .select(
      $"key".cast("string"),
      functions.from_json($"value".cast("string"), EventLog.Schema)
        .as("value")
    ).as[KeyWithValue]
    .map(eventLog => eventLog.toCleansedKeyWithValue)
```

I included the cleansing logic as a part of the case class. You can use an external function here if it's simpler.

4. Put back the remaining parts with the data classification and data deduplication:

```
.withColumn("topic", functions.when(
  ($"value.user_id" === 0 || $"value.visit_id".isNull), "invalid_data"
).otherwise("valid_data")
)
.withColumn("event_time", $"value.event_time")
.withColumn("user_id", $"value.user_id")
.withWatermark("event_time", "10 minutes")
.dropDuplicates("user_id", "event_time")
```

5. Put back the data enrichment part but this time do not write a JSON directly from it. Instead perform another cast and map operations that will read the input and integrate it to the expected output format:

```
val enrichedRecordsToWrite = recordsToWrite.join(deviceReferenceDataset,
  deviceReferenceDataset("device_type") ===
recordsToWrite("value.technical.device.type"),
  "leftOuter"
).select($"key", $"value", $"topic", $"full_name")
.as[(String, CleansedEventLog, String, String)]
.map {
  case (key, value, topic, fullDeviceName) => {
    val oldTechnical = value.technical
    val valueToReturn = oldTechnical.device.map(oldDevice => {
      value.copy(technical = oldTechnical.copy(
        device = Some(oldDevice.copy(full_name =
          Some(fullDeviceName))))
    }).getOrElse(value)
    KafkaSinkParameters(key, valueToReturn, topic)
  }
}
```

6. Select the columns to write:

```
select($"key", functions.to_json($"value").as("value"), $"topic")
```

7. Write the query results as previously.

Python

1. Read the Kafka and enrichment dataset as before:


```
input_data_stream = spark.readStream \
    .option("kafka.bootstrap.servers", "localhost:29092") \
    .option("subscribe", kafka_input_topic()) \
    .option("startingOffsets", "EARLIEST") \
    .format("kafka") \
    .load()

device_reference_dataset = spark.read.jdbc(
    url="jdbc:postgresql:bde",
    table="bde_schema.devices",
    properties={'user': 'bde_user', 'password': 'bde_password',
                'driver': 'org.postgresql.Driver'}
)
```

2. Keep the deduplication part as before except the topic resolution part based on the if-else statement. It can be removed.

3. Perform the data cleansing directly before selecting the key, value and topic columns by calling a Pandas UDF:

```
.withColumn("cleansed_value",
    mappers.clean_value(functions.col("value")))\
```

The UDF performs the data cleaning defined in separate functions:

```
@pandas_udf(MapType(StringType(), StringType()))
def clean_value(values_to_clean: pandas.Series) -> pandas.Series:
    values_to_clean_as_dict = values_to_clean.to_dict()
    values_to_return = {}
    for index, json_to_clean in values_to_clean_as_dict.items():
        json_value = json.loads(json_to_clean)
        json_value['source'] = Cleanser.cleanser_source(json_value)
        json_value['technical'] = Cleanser.cleanser_technical(
            json_value['technical'])
        topic = 'invalid_data' if json_value['user_id'] == 0 or not json_value['visit_id']
    else 'valid_data'
    values_to_return[index] = {'payload': json.dumps(json_value),
                              'device': json_value['technical']['device']['type'],
                              'topic': topic}
    return pandas.Series(values_to_return)
```

It returns not only the cleansed value but also the topic name.

4. Select the columns we need for the output generation and the data enrichment:

```
.selectExpr("key", "cleansed_value.payload AS value",
"cleansed_value.device", "cleansed_value.topic")
```

5. Join the raw data with the enrichment dataset:

```
enriched_dataset_to_write = query.join(
    device_reference_dataset,
    device_reference_dataset["device_type"] == query["device"],
    "left_outer")
```

6. Convert the combined datasets into the final output format with the help of mapInPandas:

```
data_to_write = enriched_dataset_to_write.mapInPandas(
    enrich_output_with_full_device_name,
    enriched_dataset_to_write.schema
)\
.select(
    functions.col("key"),
    functions.col("value"),
    functions.col("topic")
)
```

The mapping function combines the cleansed value column with the device full name attribute:

```
def enrich_output_with_full_device_name(input: pandas.DataFrame) -> pandas.DataFrame:
    def combine_with_full_name(value_json: str, device_full_name: str) -> str:
        json_dict = json.loads(value_json)
        json_dict['technical']['device']['full_name'] = device_full_name
        return json.dumps(json_dict)

    for rows in input:
        rows['value'] = rows.apply(lambda single_row: combine_with_full_name(
            single_row['value'], single_row['full_name']), axis=1
        )
    yield rows
```

7. Write so transformed data to Apache Kafka:

```
write_query = data_to_write.writeStream \
    .option("checkpointLocation", checkpoint_location_for_job('job4')) \
    .format('kafka') \
    .option("kafka.bootstrap.servers", "localhost:29092")
write_query.start().awaitTermination()
```

Hope it helps 🙏
Bartosz.